TECHNICAL PROJECT DEFENSE | SPEAKER PLAYBOOK

Zerodha AI Intelligence

Project Presentation & Demo Guide

A practical interview and evaluation guide for presenting the product, engineering choices, data flow, reliability, and limits.

EXPERIMENTAL INTELLIGENCE AND RESEARCH DASHBOARD

Not an official Zerodha product. No order placement or trade execution.

HOW TO USE THIS GUIDE

Contents

Use this as a speaker guide, not a slide deck. The section order follows a project defense: explain the problem, show one coherent workflow, then defend the engineering choices and limits. The scripts are written for speaking aloud; shorten or personalize them rather than reading every line verbatim.

1. Presentation story . . . 3
2. Product walkthrough . . . 4
2. Product walkthrough (continued) . . . 5
2. Product walkthrough (continued) . . . 6
2. Product walkthrough (continued) . . . 7
3. Current architecture . . . 8
4. End-to-end data flow . . . 9
5. AI architecture . . . 10
6. Reliability & Trustworthiness . . . 11
7. Limitations . . . 12
8. Neutral competitor comparison . . . 13
9. Production architecture . . . 14
10. Future scope . . . 15
11. 10-minute presentation script . . . 16
12. Three-minute quick overview . . . 18
13. Technical Q&A; . . . 19
14. What I would improve with one more month . . . 23
15. Closing tools . . . 24

The project in one sentence

An experimental intelligence layer that brings user-supplied portfolio data, deterministic analytics, on-demand market quotes, sourced news research, calculators, and natural-language interaction into one local workspace.

PROBLEM -> SOLUTION -> PRODUCT

1. Presentation story

Investors often move between a broker screen for holdings, a quote page for market prices, news sites for context, spreadsheets for risk checks, and separate calculators for planning. This project explores whether those tasks can be connected in one workflow without pretending that an AI model is a broker or a source of truth.

Positioning to repeat
The differentiator is not another stock dashboard. It is an intelligence layer that connects portfolio analytics, risk analysis, market context, news research, financial calculators, and natural-language AI interaction in one workflow.

Say this clearly
This is an experimental intelligence and research dashboard, not a trading platform. It is not an official Zerodha product and it does not place orders.

45-60 second opening script

What to say
Investors often have portfolio data, market prices, financial news, risk checks, and planning tools spread across different products. I built Zerodha AI Intelligence as a local research prototype to bring those steps into one workspace. A user can import holdings, review the extracted values, inspect portfolio and concentration metrics, request a quote, search current news with citations, use financial calculators, and ask questions in natural language. The engineering idea is to keep arithmetic in deterministic application code and use AI for explanation and research, with sources shown for current news. It is an experimental intelligence dashboard inspired by modern investment workflows - it is not an official Zerodha product and it does not execute trades. I find it interesting because it combines data validation, calculations, external APIs, search, and a user-facing AI workflow while making missing data visible.

Core narrative

• Problem: fragmented information creates context switching and makes it difficult to connect a portfolio observation to evidence.

• Solution: a single workspace that places holdings, risk snapshot, market context, news, calculators, and Q&A in sequence.

• Product: a local Next.js interface and FastAPI service, with SQLite demo persistence and selected external providers.

• Differentiator: the workflow joins portfolio-grounded questions with source-grounded market research.

• Boundary: current holdings prices are user supplied or on-demand provider quotes; there is no trading and no production account security.

TELL THE WORKFLOW, NOT A SCREEN TOUR

2. Product walkthrough

Demo sequence: start with the portfolio snapshot; show how imported data becomes reviewable holdings; move from the holdings to risk and context; finish with a question that combines the user's portfolio with cited research. Choose one example and narrate what decision-support question the screen answers.

Demo through-line
Use one holding or one sector as the thread. Show its supplied position, what the risk view can measure from a snapshot, how a quote is requested, what current sources say, and what remains unknowable without history.

1. Overview

CLICK Click Overview (or the brand/home route).

SHOW Holdings, value, invested amount, P&L and sector snapshot.

EXPLAIN This is the entry point: it answers what is in the demo portfolio and what deserves a closer look.

WHY IT EXISTS Metrics are derived from the confirmed holdings and price fields; the dashboard uses a local demo portfolio.

TECH POINT Do not imply a time-series chart is populated when historical data is not connected.

KEEP SHORT Avoid reading every number. State the portfolio size and one observation.

2. Portfolio

CLICK Choose Portfolio in the left navigation.

SHOW Position breakdown, returns, sector allocation, sorting/search and review panel.

EXPLAIN Move from a total snapshot to the individual positions behind it.

WHY IT EXISTS Position value and P&L are computed from quantity, average cost and current price; the analyst gets holdings as structured fields.

TECH POINT Prices can be user supplied. Imported values are not independently verified.

KEEP SHORT Do not call a snapshot return a historical portfolio performance series.

3. Portfolio import

CLICK Choose Import portfolio and select a CSV or PNG/JPEG/WebP screenshot.

SHOW Show the review modal before confirming. Use a clear sample CSV or screenshot.

EXPLAIN A user needs an easy path from existing broker data into analysis, but extraction must remain reviewable.

WHY IT EXISTS CSV parsing is local; screenshot extraction uses the configured Groq Vision model and sends the image to Groq. The user confirms before persistence.

TECH POINT Screenshot extraction is fallible and only captures fully visible rows. Prefer CSV for complete portfolios.

KEEP SHORT Do not upload an actual brokerage screenshot with personal data during a public demo.

GUIDED DEMO

2. Product walkthrough (continued)

4. Portfolio analytics

CLICK Open Portfolio and sort/search one position or sector.

SHOW Invested vs market value, P&L, winners and sector weights.

EXPLAIN Explain how raw holdings become useful deterministic summaries.

WHY IT EXISTS For each position: value = quantity x current price; P&L = quantity x (current price - average cost). No model is asked to do the arithmetic.

TECH POINT Unrealized P&L omits taxes, fees, dividends and cash-flow timing.

KEEP SHORT Do not turn performance ranking into investment advice.

5. Risk analysis

CLICK Choose Risk.

SHOW HHI, largest holding, top-three share, sector concentration and missing metrics.

EXPLAIN Ask what concentration risks are visible in this snapshot, and what evidence is still missing.

WHY IT EXISTS HHI is the sum of squared current-value weights. Dispersion shown is cross-sectional daily move, not annualized volatility.

TECH POINT Volatility, Sharpe, Sortino, beta, drawdown and benchmark comparison need dated return history.

KEEP SHORT Do not describe a warning threshold as a personalized risk recommendation.

6. Markets

CLICK Choose Markets; click Get quote for one symbol.

SHOW Quote source, request time, current portfolio price and the provider response.

EXPLAIN Connect a portfolio position to one on-demand quote lookup.

WHY IT EXISTS Backend requests Alpha Vantage GLOBAL_QUOTE, caches responses and applies a daily request budget and interval. Bare symbols map to .BSE in this demo.

TECH POINT Global quote data may be end-of-day; symbol coverage and entitlements vary.

KEEP SHORT Do not call it streaming, real time for every market, or an order book.

GUIDED DEMO

2. Product walkthrough (continued)

7. News

CLICK Choose News and search a specific company, sector or theme.

SHOW Show the summary and open one or two source links.

EXPLAIN Move from a portfolio question to externally sourced market context.

WHY IT EXISTS Current news runs through Groq with Browser Search enabled for GPT-OSS; result links are extracted and displayed as citations.

TECH POINT Search coverage is incomplete and citation extraction/provider availability can fail.

KEEP SHORT Do not read an uncited answer as verified. If sources are absent, say so.

8. Watchlist

CLICK Choose Watchlist; add a symbol, then request a quote if useful.

SHOW Saved local symbols and the deliberate on-demand quote button.

EXPLAIN Let a user keep a short research queue without implying quotes stream continuously.

WHY IT EXISTS Watchlist data is stored in local SQLite. Quote calls share cache, validation and provider limits.

TECH POINT This is a single local workspace, not a synchronized personal account.

KEEP SHORT Do not spend time on adding many symbols or use the watchlist as trading advice.

9. Financial calculators

CLICK Choose Calculators and change one or two assumptions.

SHOW SIP, step-up SIP, SWP, EMI, CAGR, inflation, retirement, goal planning, FD/RD or XIRR.

EXPLAIN Show how a user can model assumptions without mixing projections with forecasts.

WHY IT EXISTS Calculations are client-side formulas; XIRR uses dated cash flows and numerical iteration.

TECH POINT Outputs depend on user-entered assumptions; returns are not guaranteed.

KEEP SHORT Do not present a projected corpus as a promise or recommendation.

GUIDED DEMO

2. Product walkthrough (continued)

10. AI Analyst

CLICK Choose AI Analyst and ask: What are the largest concentration risks in this portfolio?

SHOW Answer, mode/limitations and citations on any market question.

EXPLAIN Show natural-language access to confirmed portfolio evidence and, separately, web research.

WHY IT EXISTS Portfolio questions send structured holdings to FastAPI, which calculates context before model explanation. Market questions send the query through search.

TECH POINT LLMs can be wrong; current research needs sources; portfolio price quality remains user dependent.

KEEP SHORT Do not ask a broad question that needs data the demo does not have.

11. Activity/history

CLICK Choose Activity.

SHOW The empty state and the note that conversations are not retained.

EXPLAIN Use this to explain the current persistence boundary rather than inventing an audit trail.

WHY IT EXISTS Portfolio and watchlist are stored locally; analysis conversations are not saved as activity records.

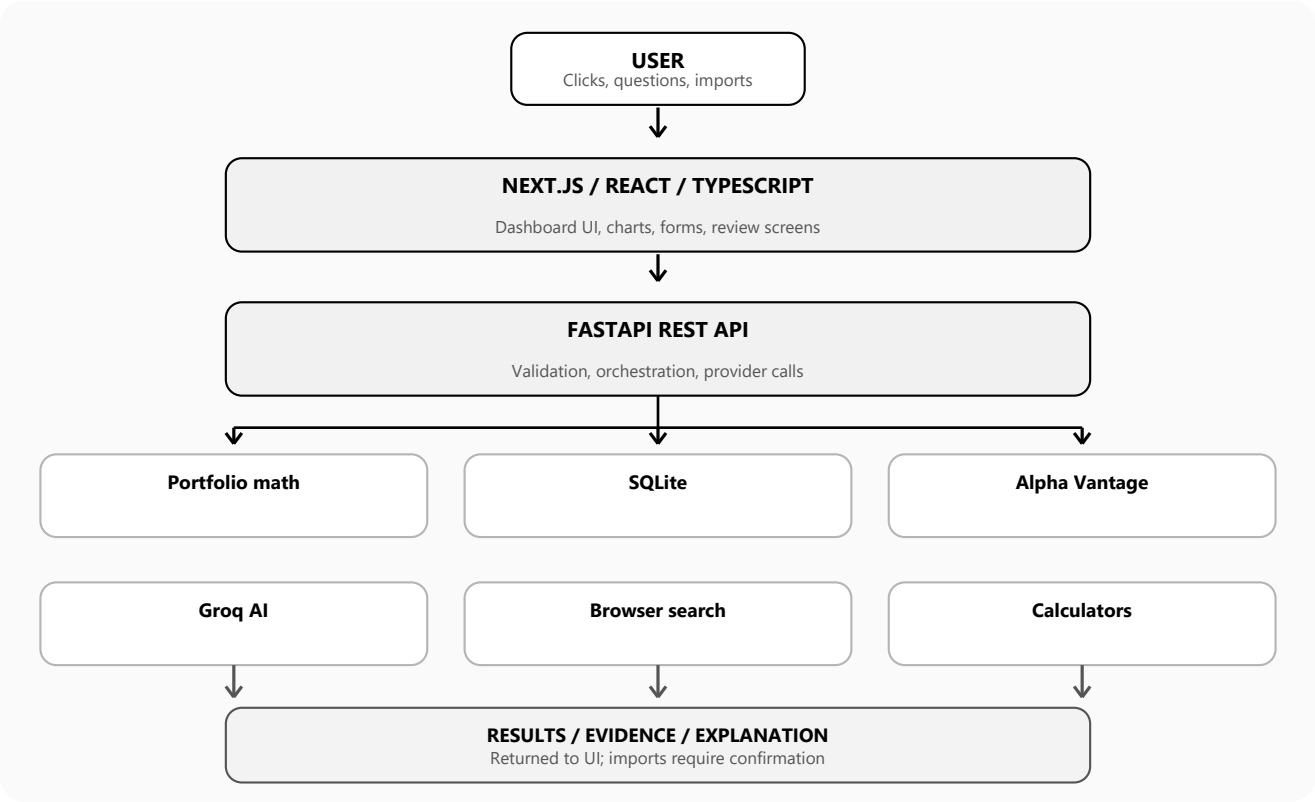
TECH POINT There is no persistent portfolio timeline or conversation history in this prototype.

KEEP SHORT Do not imply the empty state is a bug or claim historical logs exist.

WHAT EXISTS TODAY

3. Current architecture

The current system is a local single-workspace web application. The frontend calls a REST API exposed by FastAPI; Next.js proxies local /api requests to the backend during development. Provider credentials are server-side environment values, not browser constants. The diagram shows actual project boundaries at a conceptual level.



Component	Role / why it fits	Data / production improvement
Next.js + React + TypeScript	Renders the dashboard and interactive components; TypeScript describes holdings and helps catch UI integration errors.	Receives user queries, import files and API results. Production: add UI-level observability, robust session handling and accessibility review.
FastAPI REST API	Keeps provider keys and server workflows out of the browser; Python supports validation and calculation orchestration.	Validates holdings, saves demo data, calls quote and AI/search providers. Production: add auth middleware, per-user authorization, quotas and versioning.
SQLite	Low-friction persistence for a local experiment; no separate database server is needed.	Stores one shared local portfolio, watchlist, quote cache and usage counter. Production: PostgreSQL with tenant isolation, backups and migrations.
Alpha Vantage	A provider for on-demand single-symbol quote lookups.	Quote request includes symbol and server-side key; responses are cached. Production: provider abstraction, entitlement checks and multiple sources.
Groq AI + browser search	Generates explanations or retrieves current web context through a supported search workflow.	Receives portfolio context for portfolio questions, or the query for news searches. Production: model gateway, evaluation, privacy policy and provider failover.
Application calculations	Keeps core arithmetic reproducible instead of asking an LLM to calculate.	Computes value, P&L;, weights, concentration and calculator outputs. Production: unit tests, decimal policy and independently reviewed financial formulas.

TWO DIFFERENT KINDS OF QUESTIONS

4. End-to-end data flow

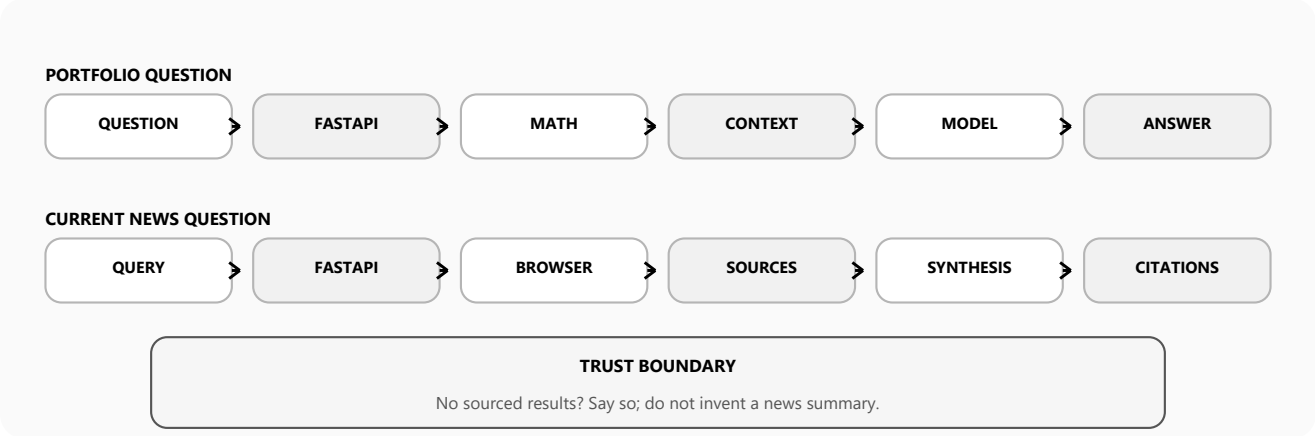
Example A - portfolio question

Question: What are the major risks in my portfolio? The browser sends the question and the confirmed holding fields to FastAPI. The service validates them and runs deterministic analytics: current values, weights, HHI, top positions and sector shares. It serializes those results into structured context for the model. The model explains the evidence and its gaps; it does not set the values. The UI returns the explanation and limitations.

- User -> Next.js UI -> FastAPI /api/v1/ai/answer
- FastAPI validates Holding objects and computes the portfolio snapshot.
- Structured summary and user-provided values -> Groq explanation
- Answer, mode, evidence and limitations -> UI
- No dated history means no reliable time-series risk statistics.

Example B - current market/news question

Question: What is happening in the Indian IT sector today? FastAPI classifies it as a market query, sends the query to the supported GPT-OSS model with Groq Browser Search enabled, receives a synthesis and tool results, extracts source URLs, and returns citations. If it has no answer or verifiable links, the safe outcome is a visible unavailable/search message rather than an invented summary.



The boundary that matters

Deterministic vs generative

Deterministic calculation: same validated inputs and code produce the same number. Generative reasoning: the model interprets context, summarizes sources and writes language; its output can vary and must be grounded and checked. Keep the number authoritative in application logic; use the model to explain it.

USE THE MODEL FOR LANGUAGE, NOT ARITHMETIC

5. AI architecture

The AI workflow has two paths. Portfolio-only questions use a computed snapshot plus structured holdings as context. Current market/news questions use a web-search-capable model and should surface source links. Screenshot import is a separate vision task that proposes rows for explicit user review.

• Portfolio path: user question + confirmed holdings -> deterministic analysis -> structured context -> AI explanation -> answer and limitations.

• News path: question -> Browser Search-enabled GPT-OSS request -> retrieved pages/results -> synthesis -> extracted links/citations -> UI.

• Image path: screenshot -> Groq Vision extraction -> candidate holdings -> review modal -> user confirmation -> local save.

• Keys remain in backend environment configuration; queries/holdings may still be transmitted to configured providers according to the selected workflow.

Why not give the LLM a calculator job?

Language models are optimized to generate plausible text, not guarantee exact floating-point arithmetic. A deterministic function can be inspected, repeated and tested. The model is useful after the calculation: it can explain that one position contributes a given amount, clarify that HHI is a concentration measure, and say that annualized volatility is unavailable because dated returns are missing.

> **Grounding rule**
> Never let a model invent a holding, price, metric, news event or source. If the structured portfolio context or retrieved source set is missing, state the gap. A prompt helps, but code validation, citations and visible limitations provide stronger controls.

Interview line

> **What to say**
> I treat the LLM as an explanation and research layer. The application owns the financial math and input validation; the model receives structured evidence, then produces language. For current claims, I enable web search and return sources. This does not eliminate hallucination, so I show sources and gaps rather than presenting the response as financial advice.
>
> **Provider reference**
> Groq's Browser Search documentation lists openai/gpt-oss-120b as supported and demonstrates enabling tools=[{"type":"browser_search"}] with tool_choice="required". This is the search path used in the current project. Groq Browser Search documentation (checked 2026-10-03).

DESIGN FOR REVIEW, EVIDENCE AND GRACEFUL GAPS

6. Reliability & Trustworthiness

Deterministic calculations

Value, P&L, allocation weights and HHI are calculated in application logic from holdings fields. Repeating the same inputs should reproduce the same outputs.

Data validation

Backend Pydantic models constrain quantities and prices. CSV and image imports show a preview before user confirmation; invalid/incomplete rows are rejected or skipped.

AI grounding

Portfolio responses are given computed context. News queries enable browser search; links are extracted. The instruction is to avoid inventing numbers, reports or sources.

Source citations

The News and Analyst surfaces can show source links returned with supported searches. No sources means the answer should be labeled unavailable or unverified.

Missing-data handling

Risk view explicitly lists volatility, Sharpe, Sortino, beta, drawdown and benchmark comparison as requiring dated return history.

Fallback behavior

When Groq is unavailable, portfolio-only questions can use deterministic, rule-based responses. Current news cannot honestly use the same fallback; it must state that search is unavailable.

No trade execution

There are no order APIs, trade placement functions or broker authorization flow. The app is informational and does not send orders.

> **Why this matters in finance**
> A polished chart can make weak inputs look authoritative. Reproducible formulas, explicit user review, dated-source links and honest missing-data messages reduce the chance that a demo output is mistaken for a verified recommendation. They do not make the prototype suitable for investment decisions.

Trust questions to answer during a demo

- Where did this number come from? Show the formula/input fields.
- What date is this quote or article? Distinguish provider retrieval time, publication time and event time.
- What can you not measure? Show the missing-history state.
- What does the model see? Explain whether the query includes portfolio context or only the news question.
- What happens when the provider fails? Show an explicit error or fallback, not a fabricated live-data state.

STATE THESE AS ENGINEERING FACTS

7. Limitations

Area	Current boundary	How to phrase it
Market data	Alpha Vantage dependency; a quote is one symbol per request, cached and rate-limited. Global data can be end-of-day and coverage/entitlement varies.	This is on-demand research data, not a tick-by-tick terminal. I show the provider and date/availability caveat.
History	The app has no reliable dated portfolio return series. Risk view does not produce time-series volatility, Sharpe, Sortino, beta, drawdown, correlation or benchmark results.	Those metrics need consistent dated returns. I mark them unavailable rather than infer them from a single snapshot.
AI and search	Models can hallucinate; search can miss reports or return weak sources; citations do not guarantee correctness.	I verify source links and separate the model's synthesis from what a source actually states.
AI advice	Not personal financial advice; no suitability review, verified account history or guaranteed result.	Treat output as research assistance. It is not a buy/sell instruction or return promise.
Security	Local single-workspace demo; no production-grade authentication, multi-tenant isolation or full security review.	Do not expose it publicly. Production needs identity, isolation, secrets controls, audit and security testing.
Persistence	SQLite holds a shared local portfolio and watchlist; conversations are not kept in activity history.	Persistence is for a one-user local demonstration, not concurrent production users.
Import	CSV is local. Screenshot extraction sends the selected image to Groq Vision; partial or blurry holdings can be missed or misread.	Review every extracted field. CSV is better for full portfolios; screenshot rows require confirmation.

Do not hide limitations

An interviewer is more likely to trust a precise boundary than an unsupported claim. Describe what the current build demonstrates, what data it lacks, and what safeguards a production version would require.

DIFFERENT PRODUCTS SOLVE OVERLAPPING PROBLEMS

8. Neutral competitor comparison

Compare categories, not named products or unverified feature checklists. Capabilities vary by platform, market, plan and account permissions. This is a positioning aid, not a claim that one category is universally better.

Capability	Broker platforms	Finance/news platforms	Portfolio analytics tools	This prototype
Trading / orders	Often core	Usually not core	Varies	No execution
Portfolio management	Account-linked	Limited or imported	Often core	Local imported snapshot
Quotes	Account/platform feed	Often broad market coverage	Varies by provider	On-demand Alpha Vantage lookup
News	Varies	Often core	Varies	Search when configured; citations expected
Risk analytics	Platform dependent	Market context more than portfolio math	Can be deep with history	Snapshot concentration; history metrics unavailable
Calculators	Varies	Varies	Varies	Several client-side planning calculators
Natural-language AI	Emerging / varies	Emerging / varies	Emerging / varies	AI analyst in one workflow
Portfolio-grounded AI	Varies	Not always account-linked	Varies	Structured confirmed holdings context
Source-grounded market research	Varies	News-first products vary	Varies	Browser search + source links when returned
Screenshot import	Broker may offer exports	Not a category guarantee	Tool dependent	Experimental vision preview + user review

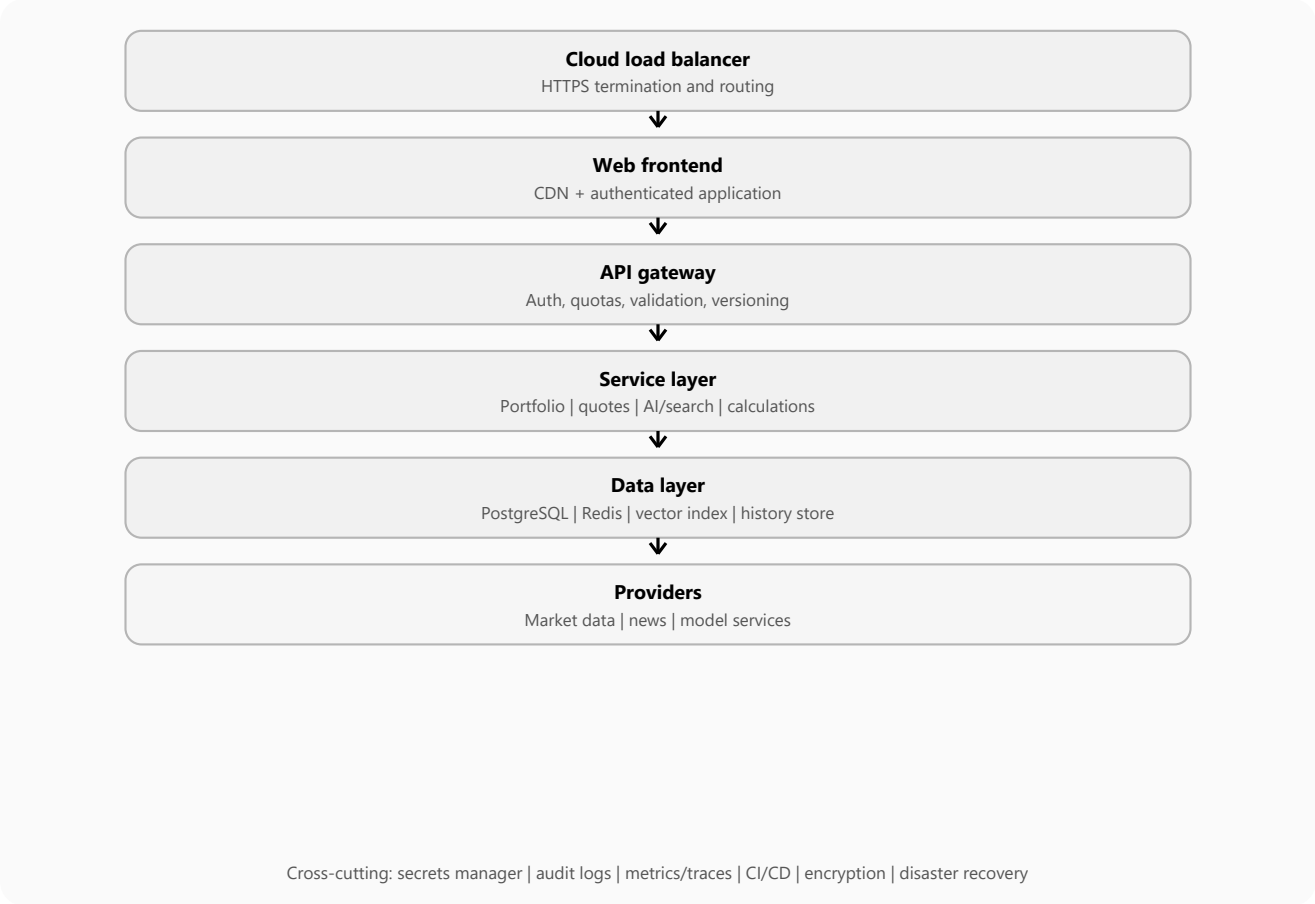
Position without overclaiming

The systems solve overlapping but different problems. This project's differentiation is an AI-first intelligence layer combining portfolio analytics, risk, market context, news and natural-language interaction in one workflow.

A PATH, NOT A CLAIM ABOUT THE CURRENT BUILD

9. Production architecture

A production design would add explicit trust boundaries around identity, provider credentials, data tenancy, request budgets and observability. The current prototype is local and single-workspace; the architecture below is a target state for discussion, not an implemented deployment.



Production controls by layer

Edge and identity

Cloud load balancer, HTTPS, API gateway, OAuth/OIDC, session lifecycle, rate limits and request-size limits.

Service layer

Containerized FastAPI services split by portfolio, market data, search and calculations; background tasks for ingestion and provider retries.

Data layer

PostgreSQL for user-owned records; Redis for cache/quotas; object storage for uploads; vector store only when a governed document RAG use case exists.

Security and governance

Secrets manager, encryption at rest and in transit, per-user authorization, data retention rules, audit logs, key rotation and threat modeling.

Operations

Structured logs, metrics, traces, provider health, alerting, CI/CD, migration checks, rollback, backups and disaster-recovery tests.

Quality

Automated calculation tests, import fixtures, API contract tests, model evaluation sets, citation checks, prompt/version tracking and human review for high-impact changes.

PRIORITIZE TRUSTWORTHY DATA BEFORE MORE AI

10. Future scope

1. Data foundation
Historical price and portfolio transactions pipeline; reliable timestamps, corporate actions, source status and data-quality checks.

2. Risk analytics
Annualized volatility, Sharpe/Sortino, beta, drawdown, correlation matrix and benchmark comparison built from dated returns.

3. Research corpus
RAG over filings, earnings calls and trusted financial documents with document dates, chunk provenance and citation validation.

4. Product security
Authentication, OAuth, tenant isolation, encryption, secrets management, API quotas, audit trail and a cloud deployment.

5. AI quality
Evaluation set for factuality, numerical consistency, refusal on missing data, source coverage, citation precision and regression across model/prompt changes.

6. Operations
Monitoring, tracing, dashboards, provider error budgets, CI/CD, containers, backups and incident playbooks.

7. Personalization
User-controlled research preferences, watchlist context and explainable alerts with provenance; opt-in and reversible.

Keep automated trading separate
Order execution introduces credential custody, account authorization, suitability, market access, regulatory/compliance obligations, idempotency, reconciliation and high-availability requirements. It is a distinct safety-critical system, not a small extension of a research dashboard.

TIME-BOXED, CONVERSATIONAL

11. 10-minute presentation script

0:00-1:00 | Problem

Investors often move between a broker for holdings, a quote source, news sites, spreadsheets and separate calculators. The hard part is not only seeing data; it is connecting a portfolio fact to market context and knowing which conclusions the data can support. A portfolio value alone does not tell me whether the largest holding dominates, whether a price is current, or whether there is enough history for a risk statistic. This project explores that workflow in one local workspace. I will be clear about the boundary: it is an experimental research dashboard, not a trading product, and it is not officially affiliated with Zerodha. My goal is to explain engineering choices and evidence limits, not make an investment pitch.

1:00-2:00 | Product overview

The overview summarizes a supplied portfolio and points to deeper analysis. The workflow is import, review, analyze, inspect risk, request market context, search news, then ask a question. I will follow one example through those steps instead of listing every screen. The differentiator is the intelligence layer across the workflow, rather than one more price chart. Each view answers a related question: what do I hold, where is value concentrated, what quote context is available, and what do sources say? This is a single-user local demo with no production authentication. It is designed to make a research workflow understandable, not replace a broker or market terminal.

2:00-3:30 | Portfolio

Here I can see positions, invested value, current value and unrealized P&L. These numbers come from code: quantity times cost or price. I can import a CSV locally or choose an image for a vision-assisted preview. The review step matters: I verify symbol, quantity, average cost and current price before confirming because CSV headers can vary and vision can misread digits. For a complete portfolio I prefer CSV; one screenshot shows only the rows visible in that image, and a cropped row should not be treated as complete. I can trace one holding's quantity and prices into value and P&L. These are snapshot calculations from supplied fields; they do not include taxes, fees, dividends or cash-flow timing. The model does not calculate these values for me.

3:30-4:30 | Risk

The risk view focuses on what a snapshot can support: largest position, top-three weight, sector concentration and HHI. HHI squares each position's current-value share and sums those squares, so higher values mean value is concentrated in fewer positions. This can start a discussion about exposure, but it is not a full risk model or personalized advice. The page separates cross-sectional daily-move dispersion from historical volatility. I do not claim historical volatility, Sharpe, Sortino, beta, correlation, drawdown or benchmark performance because this build lacks a dated portfolio return series. The missing-data panel is intentional: it explains what extra data would be required.

4:30-5:30 | Market data

For a holding, I can request a quote on demand. The backend calls Alpha Vantage, caches the response and applies a local request budget, reducing repeated provider calls. The demo maps bare symbols to a BSE suffix, so I still verify symbol coverage instead of assuming every listing resolves. I will check the quote date before describing it. Global quote data may be end-of-day, and coverage or entitlement varies. This is not streaming, a complete market feed or a source of guaranteed current prices. It does not send trades. If the provider is unavailable or the budget is used, the UI should say so rather than imply that a fresh quote was retrieved.

10-minute presentation script (continued)

5:30-6:30 | News

News search takes a query through the backend and enables web search through the configured Groq workflow. The result should include source links, and I will open one to confirm its publication date and the claim it supports. I distinguish the date an article was published from the date of the event it describes. Search can miss sources, and a citation does not guarantee a source is complete or correct. If the provider returns no sources, the app should show that limitation instead of making up a news summary. For an important claim, I would compare sources and read the underlying report rather than treat a short AI synthesis as the final word.

6:30-8:00 | AI Analyst

For a portfolio question, the frontend sends structured confirmed holdings to FastAPI. The backend calculates a snapshot, then gives the model that structured context. If I ask about concentration, position weights and HHI are computed first; the model explains them and notes missing history. The model explains; application code owns the arithmetic. For current market questions, the query uses browser search and the app returns links when available. These are separate paths: portfolio questions can use confirmed holdings, while current news depends on retrieval and citations. This reduces arithmetic errors and lets the user inspect external evidence, but it does not eliminate hallucination. I would verify important claims, and the app should not present an unsupported news summary as fact.

8:00-9:00 | Architecture

The frontend is Next.js, React and TypeScript. It calls a FastAPI REST backend. The backend validates fields, calculates metrics, stores the local demo portfolio and watchlist in SQLite, and calls Alpha Vantage or Groq when requested. During development, Next.js proxies API paths to the local backend; provider keys stay on the server side rather than browser code. SQLite is a fast local choice, not a multi-user production database; the current storage is a shared demo workspace. These boundaries are useful even in a prototype: the UI handles interaction, the API validates and coordinates, deterministic functions calculate, and providers return quotes or model responses. Production would add identity, user isolation, PostgreSQL, secrets management, encryption, monitoring and audit logs.

9:00-10:00 | Reliability, limits, differentiation

The reliability choices are deterministic calculations, user review for imports, citations for news, visible gaps when history is missing, and no trading execution. These matter because a polished interface can make uncertain inputs look authoritative. The biggest limits are local single-workspace security, provider quotas and no historical return database. Screenshot extraction can misread and sends the selected image to the configured vision provider; structured CSV is preferable for complete holdings. Compared with broker, news and analytics products, this prototype addresses a different overlap: a portfolio-grounded intelligence workflow. The systems solve overlapping but different problems. I am not claiming this is better or production ready. I am showing how portfolio analytics, risk, market context and research can connect, and which controls must improve before real users depend on it.

WHEN ASKED: GIVE ME A QUICK OVERVIEW

12. Three-minute quick overview

Three-minute script

I built Zerodha AI Intelligence as an experimental local dashboard for investors who currently move between portfolio data, quotes, news, risk checks and calculators. The idea is to connect those steps in one research workflow. It is inspired by investment-platform workflows, but it is not an official Zerodha product and it does not place trades. The user can import a CSV or a screenshot, inspect a review preview, and confirm holdings. Portfolio and risk pages calculate position value, P&L;, sector weights and concentration from structured values. The market page can request an Alpha Vantage quote on demand. The News and AI Analyst pages accept natural-language questions; current-news requests use web search and should surface source links. The calculators model user-supplied assumptions such as SIP, EMI, CAGR or XIRR. In a demo I would trace one holding through the workflow: first show its imported quantity and prices, then point to the calculation, then ask what concentration says and what it cannot say. After that I would request a quote and search a current sector question, opening a source link to inspect the underlying report. This keeps the demo focused on how information connects, not on clicking every control. If data or sources are missing, I would show that state instead of improvising an answer. Technically, the frontend is Next.js, React and TypeScript. It talks to a FastAPI REST backend. The backend validates data, calculates analytics and coordinates SQLite persistence plus external API requests. Provider keys stay server-side. The important design choice is that code computes the financial numbers and the model explains or researches them. The prototype is deliberately honest about gaps. Prices can be user supplied, quote data can be delayed, screenshot extraction can misread, AI can hallucinate, and the app has no production authentication or multi-user isolation. Historical risk measures like Sharpe, beta and drawdown need dated returns that this build does not have. My next priorities would be a historical data pipeline, tested risk metrics, production identity and isolation, then an AI evaluation suite. The differentiator is an intelligence layer connecting portfolio, risk, market context and research - not trade execution.

INTERVIEW-READY ANSWERS

13. Technical Q&A

Keep answers direct. State what is implemented first, then the trade-off, then one production improvement. If you have not benchmarked a provider or deployed at scale, say that rather than imply measured results.

Architecture

Architecture | Why FastAPI?

FastAPI gives the project a small typed REST layer with request validation and async support for external calls. Python also fits the calculation and data workflow. I chose it for prototype speed and a clear API boundary, not because it is automatically the best production framework. I would add auth, observability and contract tests before scaling it.

Architecture | Why Next.js?

Next.js gives the React interface a familiar application structure and a local API rewrite path, while TypeScript helps keep holding shapes consistent between components. This dashboard does not depend on server rendering for its core value, so a simpler React SPA could also work. The framework choice is pragmatic, not a product claim.

Architecture | Why SQLite?

SQLite is simple to run locally and avoids a separate database service for a single-workspace demo. The current storage is explicitly shared and local. It is not appropriate for concurrent tenants or production account isolation. I would migrate to PostgreSQL with schema migrations, user ownership, backups and access controls.

Architecture | Why separate frontend and backend?

The browser should not hold provider credentials, and a backend is the right place for validation, rate limits, persistence and orchestration. The REST boundary also makes it easier to replace the UI or provider later. Production would add authenticated user identity and authorization at every resource boundary.

Architecture | How does a request travel?

The UI sends a JSON or multipart request to FastAPI through the local Next proxy. FastAPI validates the payload, computes or retrieves context, calls a provider only when needed, and returns structured JSON with result modes, evidence or limitations. The frontend renders the result and exposes source links where available.

Backend

Backend | How do you validate imported data?

CSV parsing maps aliases for symbol, quantity, average price and current price, then validates rows through the Holding model. Screenshot extraction proposes rows and the UI shows a review modal. The user confirms before persistence. That is a human checkpoint, not a guarantee of OCR accuracy, so the user must inspect the values.

Backend | Why SQLite local storage?

It keeps the demo reproducible with minimal setup. The holdings and watchlist are stored locally; a quote cache and daily request counter also live there. The current key is a shared local demo identity, so it is not multi-user safe. A production schema would require ownership and transaction design.

Backend | How are failures handled?

Provider calls have timeouts and return clear error states; portfolio-only questions have a deterministic fallback when Groq is unavailable. News is different: if search or citations are unavailable, the app should not invent a current summary. I would add structured error codes, retry policy and tracing so operators can distinguish quota, credentials, model and network failures.

Backend | How would you prevent duplicate quote calls?

The app checks a cache before requesting and rechecks inside an async lock, then tracks per-day usage and a minimum interval. That is adequate for a single process. Multiple production workers need a shared Redis lock or atomic quota service because an in-memory lock does not coordinate across processes.

Frontend

Frontend | Why TypeScript?

The domain has repeated data shapes such as a Holding with symbol, quantity, average cost and current price. TypeScript helps catch mismatched fields during development and makes component interfaces clearer. It does not replace server validation; the backend remains the trust boundary because browser payloads are untrusted.

AI/LLM

AI/LLM | How does the AI understand portfolio context?

For portfolio questions, the frontend sends confirmed holdings as structured fields. FastAPI computes values, P&L, sector allocation and concentration, then supplies a bounded summary and data-quality caveats with the prompt. The model does not query SQLite on its own and does not receive unrelated account access.

AI/LLM | How do you prevent hallucinations?

I cannot guarantee that a language model never hallucinates. I reduce risk by computing numbers in code, keeping prompts evidence-focused, using browser search for current claims, extracting source links, and returning unavailable states when evidence is missing. The user still needs to inspect source quality and verify important information.

AI/LLM | Why not use an LLM for portfolio math?

LLM output is probabilistic and not a reliable calculator. A deterministic function makes formulas inspectable, testable and reproducible. The model can translate calculated values into a readable explanation, but the value itself should come from validated input and application logic. That separation is especially important when money-related numbers are involved.

AI/LLM | How does News search work?

For a current-market query, FastAPI uses the configured GPT-OSS model with Groq Browser Search explicitly enabled and required. The provider executes search and returns a synthesis plus tool results; the backend extracts source URLs and the UI displays them. Without links or an answer, the app reports search as unavailable.

AI/LLM | How would you evaluate answer quality?

I would build a versioned test set of portfolio and news questions with expected numeric evidence, required refusals, trusted source examples and known missing-data cases. Metrics would include numeric consistency, citation precision, source coverage, unsupported-claim rate, latency and cost. Human review remains important for ambiguous news and safety-sensitive outputs.

Prompt engineering

Prompt engineering | What is the prompt's role?

The prompt describes the analyst role, boundaries and evidence policy. It tells the model not to invent values, to distinguish user-supplied prices from sourced news and to mention limitations. Prompts improve behavior but are not security controls. Validation, tool constraints, typed outputs, tests and UI disclosures provide separate safeguards.

Portfolio analytics

Portfolio analytics | How does HHI work?

For each holding, compute its share of total current portfolio value. Square each share and sum them: HHI = sum of weight squared. A portfolio concentrated in a few large positions has a higher value than one spread across many similar-sized positions. It is a snapshot concentration indicator, not a complete risk score.

Portfolio analytics | Why not calculate Sharpe reliably?

Sharpe needs a time series of portfolio returns, a defined period, a risk-free rate and consistent sampling. This project has a snapshot of holdings and prices but no dated portfolio history. Calculating a time-series statistic from a single observation would be misleading, so the interface marks it as unavailable.

Portfolio analytics | What does P&L mean here?

The current snapshot uses quantity times current price minus quantity times average cost. It is an unrealized position-level estimate from supplied values. It does not reconstruct cash flows, fees, taxes, dividends, corporate actions or deposits. It should not be confused with a broker statement or verified return history.

Reliability

Reliability | How do you handle API limits?

The quote endpoint caches results, counts daily requests and enforces a minimum interval. It reports quota failures instead of claiming fresh data. The local limits are configuration values and must match the provider plan. In production I would centralize quotas, add backoff, provider health metrics and user-level fairness.

Reliability | What is the fallback when AI is unavailable?

Portfolio questions can use deterministic rule-based summaries because the data is already structured. News requires current external evidence, so a local fallback must not invent current events. It should explain that search is unavailable and preserve any relevant provider detail for the user or operator.

Security

Security | How are API keys protected?

Keys are read by the backend from environment configuration and should never be placed in a NEXT_PUBLIC variable or checked into source control. The current app is a local demo, not a hardened secret-management system. Production would use a secret manager, key rotation, least privilege, audit logs and restricted deployment access.

Security | Is the app multi-user secure?

No. The current demo has no login boundary and its local store is a shared workspace. It is intended to run locally, not be exposed publicly. Production needs authentication, authorization checks on every record, per-user data isolation, encryption, session protection, rate limiting and a security review.

Scalability

Scalability | How would you scale to 100,000 users?

I would first redesign tenancy and operations: stateless API services behind a gateway, PostgreSQL with user-scoped data, Redis for cache and quotas, a queue for ingestion, and separate provider adapters. Then I would load-test and monitor bottlenecks. I would not claim the SQLite demo already scales to that level.

Scalability | Why Redis and a vector database?

Redis could coordinate cache, quotas, distributed locks and short-lived work across API instances. A vector database is only useful if the product retrieves relevant chunks from filings, earnings calls or other long documents. It should not be added as decoration; document provenance, access controls and retrieval evaluation must come first.

Limitations

Limitations | What is the biggest limitation?

The largest product limitation is the absence of reliable historical account and market data. Without dated portfolio values, it cannot support serious time-series risk analytics or benchmark comparisons. The largest deployment limitation is security: it is a local single-workspace demo with no production authentication or tenant isolation.

Limitations | Are quotes real time?

Not as a general promise. Quotes are requested on demand through Alpha Vantage and global quote responses may be end-of-day depending on symbol and entitlement. I show provider status and avoid describing the page as streaming. A production product would need explicit timestamps, exchange coverage and entitlement validation.

Product decisions

Product decisions | Why require user confirmation after import?

CSV columns can be mapped incorrectly and OCR can confuse symbols or digits. A preview gives the user a chance to catch errors before those values drive analytics or AI explanations. Confirmation is a practical human-in-the-loop checkpoint; it is not proof that every value is correct.

Product decisions | Why support screenshot import?

It reduces friction when a user only has a visual broker screen, but it is a convenience path rather than the most accurate import. Vision can miss cropped rows and sends the selected image to Groq. For complete portfolios, a structured CSV export is preferable.

Product decisions | What makes it different from existing platforms?

The project combines a portfolio snapshot, concentration analysis, on-demand quotes, sourced search, calculators and portfolio-grounded natural-language questions in one workflow. Those capabilities overlap with other categories, so I do not claim superiority. The prototype explores how an AI-first intelligence layer can connect them.

Product decisions | Why no automated trading?

Order execution adds a different risk class: account credentials, authorization, order validation, idempotency, settlement, auditability, compliance and failure recovery. The current goal is research and explanation. A trading service should be separately designed, reviewed and authorized rather than silently attached to an experimental AI interface.

Production

Production | How would you productionize it?

I would start with auth, tenant isolation, a provider-neutral service layer and PostgreSQL. Then add HTTPS, secrets management, encryption, quotas, audit logs, monitoring, CI/CD, backups and security testing. I would build historical data ingestion and calculation tests before advertising advanced risk metrics, then evaluate AI and citation quality.

Production | What does RAG add?

RAG can retrieve relevant passages from filings, earnings calls and trusted financial documents and give the model attributable context. It helps only when the corpus is licensed, current, indexed with metadata and evaluated for retrieval quality. It does not make the model automatically correct; source dates and citations still matter.

PREPARED ANSWER

14. What I would improve with one more month

What to say

With another month, I would start with a historical data pipeline because trustworthy history unlocks the risk features the product currently labels unavailable. I would implement and test volatility, Sharpe, Sortino, beta, drawdown, correlation and benchmark comparison only after validating the dated return series. In parallel, I would add production authentication and multi-user isolation, then move persistence to PostgreSQL and put secrets, encryption, rate limits and audit logs behind proper services. I would improve market-data coverage, add RAG over selected filings and earnings calls with citation provenance, and create an automated AI evaluation set for numeric consistency, source quality and refusal behavior. I would deploy with containers and CI/CD, then add monitoring and tracing. I would keep automated trading out of scope because it has separate security, compliance and reliability requirements.

Priority order

- Historical data pipeline and quality controls.
- Advanced risk metrics with reference tests and independent validation.
- Production authentication, OAuth, user isolation and auditability.
- Better market-data providers and explicit timestamps/entitlements.
- RAG over licensed financial documents with source provenance.
- AI evaluation framework for factuality, citations, refusals and regressions.
- Cloud deployment, containerization, CI/CD, monitoring and logging.
- Multi-user database architecture and incident/backup practices.

ONE-MINUTE ANSWER AND LAST-MINUTE CHECKLIST

15. Closing tools

One-minute final explanation

About 60 seconds

Zerodha AI Intelligence is an experimental local research dashboard that connects a portfolio snapshot with concentration analysis, on-demand quotes, news search, financial calculators and natural-language questions. The frontend is Next.js, React and TypeScript; FastAPI validates requests, runs analytics, stores local demo data and coordinates external providers. Core numbers such as value, P&L;, sector weights and HHI are calculated deterministically. AI is used to explain structured portfolio evidence or synthesize current search results with source links. Imports remain reviewable before saving. The main boundaries are no production authentication, no multi-user isolation, limited historical data, provider quotas and the possibility of model or source error. It is not an official Zerodha product and it does not place trades. The differentiator is an intelligence layer that joins portfolio, risk and research in one workflow.

Last-minute checklist

- Application running; correct folder and browser tab prepared.
- FastAPI health endpoint returns successfully.
- Backend API key configuration present and working; never show key values on screen.
- Alpha Vantage quote flow checked without exhausting the request budget.
- AI Analyst tested with one portfolio question; News search tested and citation links open.
- Demo portfolio loaded with numbers you can explain.
- CSV ready; screenshot ready if demonstrating image import.
- Use synthetic or approved sample data; avoid exposing real account details.
- Backup screenshots available in case a provider is unavailable.
- Architecture diagram open and production diagram clearly labeled as future design.
- Limitations memorized: no trades, no production auth, no reliable history metrics.
- 30-second project explanation memorized; opening and closing scripts rehearsed.

Final reminder

Do not claim official Zerodha affiliation, order execution, guaranteed AI correctness, live prices for every market, or unavailable historical analytics. Keep the demo focused on engineering decisions and evidence boundaries.